Attention and mask

colab-dl8.ipynb

Paulo Vieira

Agenda

Ferramentas de visualização de Attention

Attention

Tipos de Attention

Mask

Tipos de Mask

Attention

1. O que é Attention?
Attention é um mecanismo que permite ao modelo:

Dar mais importância às partes relevantes do input

Exemplo
Frase:
"O gato sentou-se no tapete."
Para entender **“sentou-se”**, o modelo presta mais atenção a: “gato” (quem fez a ação)

Ideia base

- Cada palavra olha para as outras
- Atribui “pesos” (importância)
- Combina informação

2. Tipos de Attention

Tipo de Attention	O que faz	Exemplo
Self-attention	Cada token olha para todos os outros	BERT
Causal attention	Só olha para o passado	GPT
Cross-attention	Liga input ↔ output	Tradução (T5, BART)
Multi-head attention	Várias attentions em paralelo	Todos os Transformers

Resumo dos tipos

- **Self-attention** → compreensão
- **Causal** → geração
- **Cross** → ligação entre sequências
- **Multi-head** → múltiplas perspetivas

Ferramentas de visualização de Attention

BertViz é uma ferramenta para visualizar attention em modelos Transformer

O ! serve para executar **comandos do sistema (shell)** dentro do notebook

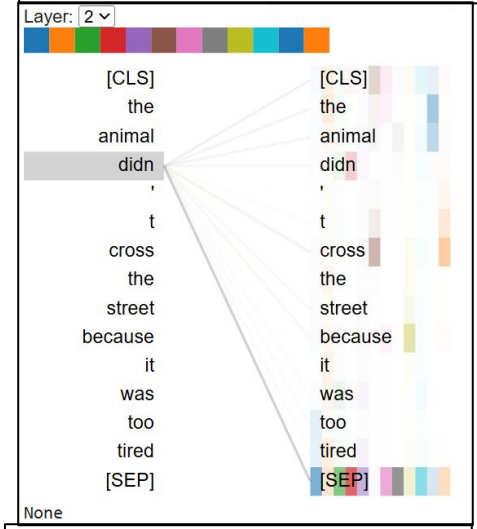
```
!pip install bertviz transformers
```

Attention for default, self-attention

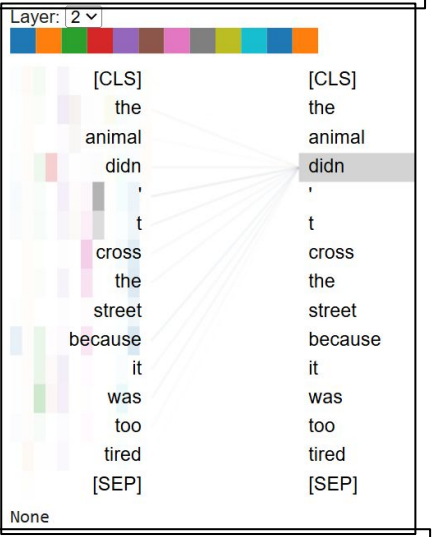
```
from transformers import AutoModel, AutoTokenizer
from bertviz import head_view
from IPython.display import display

model = AutoModel.from_pretrained("distilbert-base-uncased", output_attentions=True)
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

sentence = "The animal didn't cross the street because it was too tired"

inputs = tokenizer(sentence, return_tensors="pt")
outputs = model(**inputs, output_attentions=True)

tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

display(head_view(outputs.attentions, tokens))
```

Tipos de Attention: self-attention

self-attention (bidirectional) da head -2

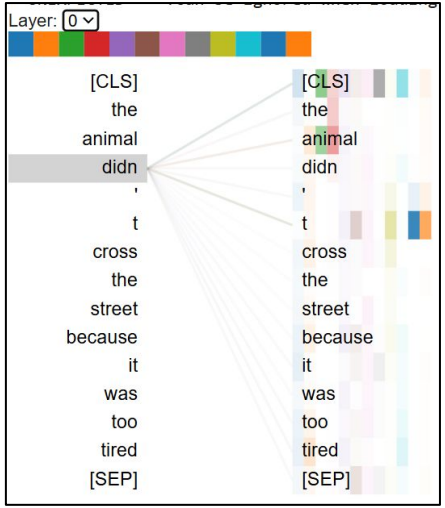


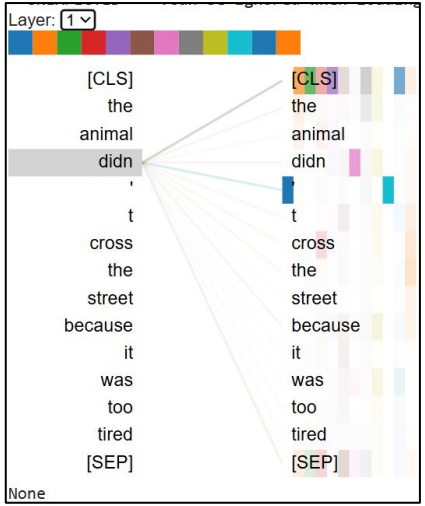
O token didn olhando para a frente e para trás (neste sentido →)
Olhar para trás de didn: conectado a tokens acima
Olhar para a frente de didn: conectado a tokens abaixo

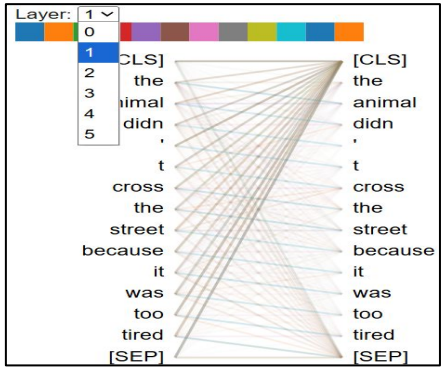


O token didn olhando para a frente e para trás (neste sentido ←)
Olhar para trás: conectado a tokens abaixo
Olhar para a frente conectado a tokens acima

multi-head (6 heads)

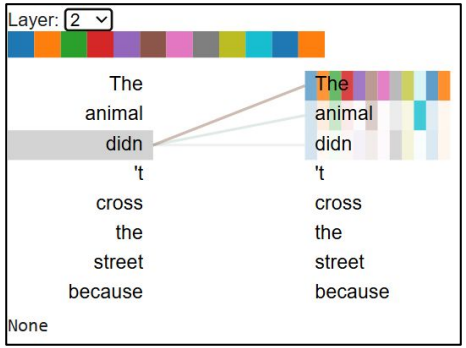


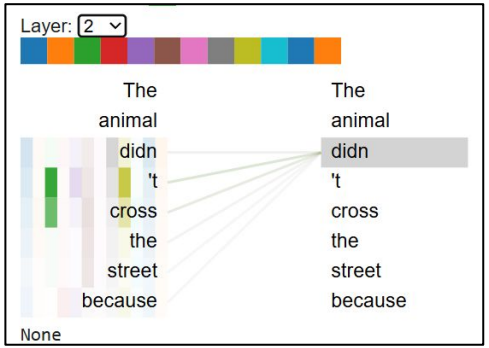




Tipos de Attention: causal-attention

causal-attention da head -2





O token didn olhando para trás (neste sentido →)
Olhar para trás :conectado a tokens que estão acima

O token didn olhando para trás (neste sentido ←)
Olhar para trás: conectado a tokens que estão abaixo

```
from transformers import AutoModelForCausalLM,
AutoTokenizer
from bertviz import head_view
from IPython.display import display

model = AutoModelForCausalLM.from_pretrained( "gpt2",
output_attentions= True)
tokenizer = AutoTokenizer.from_pretrained( "gpt2")

inputs = tokenizer( "The animal didn't cross the street
because", return_tensors= "pt")

# IMPORTANTE: garantir attentions
outputs = model(**inputs, output_attentions= True)

# converter tokens
tokens =
tokenizer.convert_ids_to_tokens(inputs[ 'input_ids'][0]
)

# visualizar
display(head_view(outputs.attentions, tokens))
```

Tipos de Attention: cross-attention

Cross-attention:

estás a ver **decoder → encoder attention**

Como interpretar visualmente

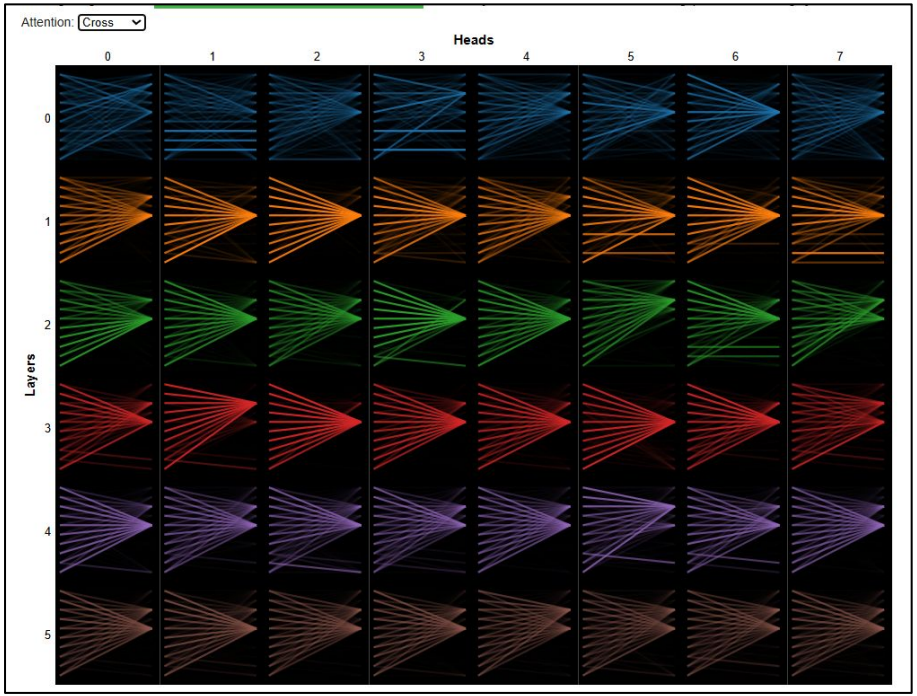
Estrutura

- **linhas horizontais** → layers (0 a 5)
- **colunas** → heads (0 a 7)

Cada quadrado = 1 head numa layer

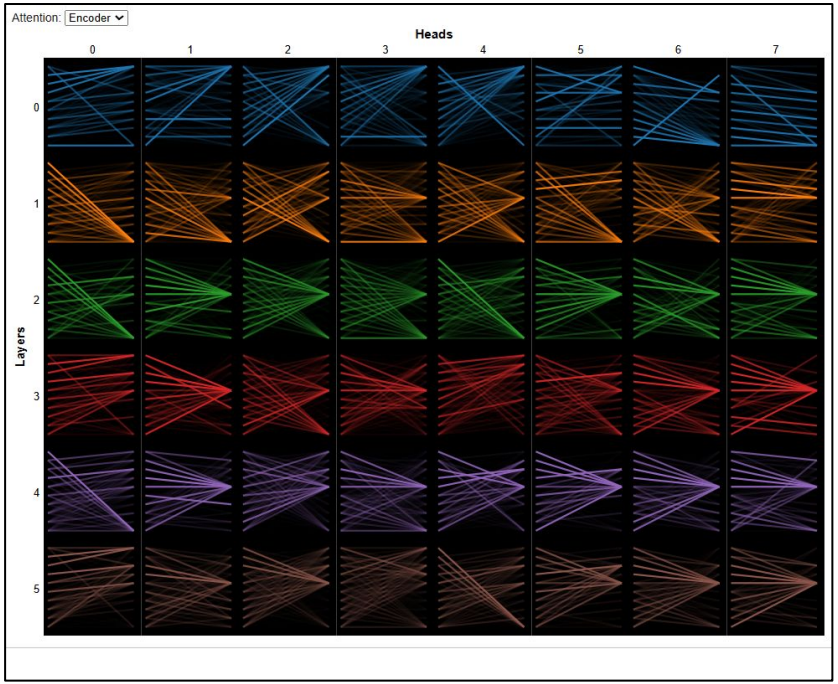
O que é cross-attention

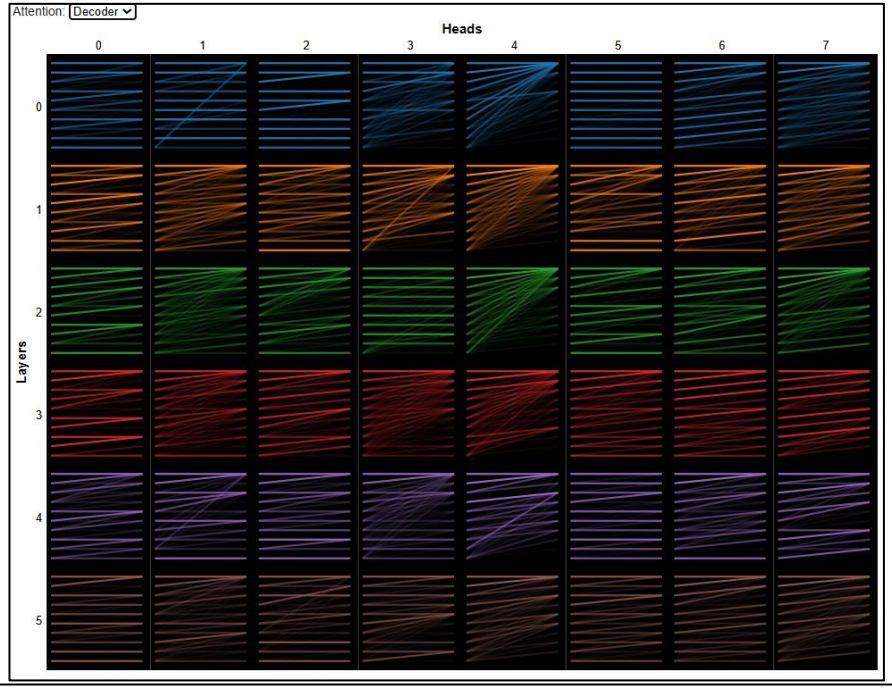
Cada linha dentro de um quadrado representa: um token do **decoder (output)** a olhar para tokens do **encoder (input)**




```python
from bertviz import model_view
from transformers import T5Tokenizer, T5ForConditionalGeneration

model = T5ForConditionalGeneration.from_pretrained("t5-small",
output_attentions=True)
tokenizer = T5Tokenizer.from_pretrained("t5-small")

input_text = "translate English to German: The house is wonderful"
inputs = tokenizer(input_text, return_tensors="pt")

# NÃO usar generate
outputs = model(**inputs, decoder_input_ids=inputs["input_ids"],
output_attentions=True)

# tokens
encoder_tokens =
tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])
decoder_tokens =
tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

# visualizar
model_view(
    encoder_attention=outputs.encoder_attentions,
    decoder_attention=outputs.decoder_attentions,
    cross_attention=outputs.cross_attentions,
    encoder_tokens=encoder_tokens,
    decoder_tokens=decoder_tokens
)
```

Tipos de Attention: self-attention





Self-attention no encoder: (entendimento)
Cada token do input olha para os outros tokens do input

Self-attention (causal, com mask causal) no decoder: (geração)
No decoder, a self-attention é causal: cada token t só pode atender a tokens com posição ≤ t.

Masks

- O que é Mask?
- Mask é um mecanismo que diz ao modelo: “Aqui NÃO podes olhar”

Ideia simples

- Attention → onde olhar
- Mask → onde não olhar

Tipos de Mask		
Tipo de Mask	Para que serve	Usado em
Padding mask	Ignorar [PAD]	BERT, todos
Causal mask	Impedir ver o futuro	GPT
Encoder padding mask (em Cross-attention)	Ignorar partes do input	T5, BART
Custom mask	Definir regras próprias	Investigação

Relação entre Attention e Mask	
Attention	Mask associada
Self-attention	Padding mask
Causal attention	Causal mask
Cross-attention	Cross mask

Resumo final (muito importante)

- **Attention**
  → decide *quanto olhar*
- **Mask**
  → decide *se pode olhar*

Padding → ignora lixo
Causal → bloqueia futuro
Cross → ignora padding do input

Tipos de Masks: padding mask

```
# padding mask com pytorch
# Ignora tokens [PAD]
import torch
import torch.nn.functional as F

# Scores (attention)
scores = torch.rand(1, 4, 4)

# Mask: últimos 2 tokens são padding
mask = torch.tensor([[1, 1, 0, 0]])

# Ajustar dimensão
mask = mask.unsqueeze(1)

# Aplicar mask
scores = scores.masked_fill(mask == 0, float('-inf'))

weights = F.softmax(scores, dim=-1)
```

```
# padding mask com HF
from transformers import AutoTokenizer

tokenizer =
AutoTokenizer.from_pretrained("bert-base-uncased")

inputs = tokenizer(
    ["Hello world", "Hi"],
    padding=True,
    return_tensors="pt"
)

print(inputs["attention_mask"])
```

Output:

```
tensor([[1, 1, 1, 1],
        [1, 1, 1, 0]])
```

Tipos de Masks: causal mask; impede ver o futuro

```
# usando pytorch
import torch
seq_len = 5

# triangular inferior
mask = torch.tril(torch.ones(seq_len, seq_len))

print(“mask:”,mask)
```

Aplicar no attention:

```
scores = torch.rand(5, 5)
scores = scores.masked_fill(mask == 0, float('-inf'))
```

mask:
1 0 0 0 0
1 1 0 0 0
1 1 1 0 0
1 1 1 1 0
1 1 1 1 1

```
... tensor([[0.3281, 0.1195, 0.8378, 0.1336, 0.7998],
            [0.6876, 0.5118, 0.0276, 0.2715, 0.2298],
            [0.3586, 0.0539, 0.2321, 0.4156, 0.1516],
            [0.5401, 0.7012, 0.3164, 0.9889, 0.6889],
            [0.6235, 0.7680, 0.7116, 0.8146, 0.7469]])
    tensor([[0.3281,   -inf,   -inf,   -inf,   -inf],
            [0.6876, 0.5118,   -inf,   -inf,   -inf],
            [0.3586, 0.0539, 0.2321,   -inf,   -inf],
            [0.5401, 0.7012, 0.3164, 0.9889,   -inf],
            [0.6235, 0.7680, 0.7116, 0.8146, 0.7469]])
```

```
# usando HF
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained("gpt2")
tokenizer = AutoTokenizer.from_pretrained("gpt2")

inputs = tokenizer("Hello world", return_tensors="pt")

outputs = model(**inputs)
```

causal mask é aplicada automaticamente

Tipos de Masks: Cross-Attention Mask

```
# ignora o padding [PAD] no encoder, em T5 é zero
# exemplo com T5
from transformers import T5Tokenizer, T5ForConditionalGeneration

tokenizer = T5Tokenizer.from_pretrained("t5-small")
model = T5ForConditionalGeneration.from_pretrained("t5-small")

inputs = tokenizer(
    [
        "translate English to German: The house is wonderful",
        "translate English to German: Hi",
        "translate English to German: This is a very long sentence for testing"
    ],
    padding=True,
    return_tensors="pt"
)
print(inputs)
# attention_mask do encoder
print(inputs["attention_mask"])
outputs = model(**inputs, decoder_input_ids=inputs["input_ids"])
```

```
# exemplo explicito (manual cross attention)
encoder_mask = torch.tensor([[1, 1, 1, 0]])  # último token é PAD

scores = torch.rand(4, 4)
print(scores)
scores = scores.masked_fill(encoder_mask == 0, float('-inf'))
print(scores)
```

```
tensor([[0.2920, 0.0575, 0.5778, 0.6790],
        [0.7727, 0.0110, 0.3387, 0.2174],
        [0.9370, 0.6427, 0.2782, 0.9343],
        [0.8520, 0.6040, 0.5259, 0.2708]])
tensor([[0.2920, 0.0575, 0.5778,   -inf],
        [0.7727, 0.0110, 0.3387,   -inf],
        [0.9370, 0.6427, 0.2782,   -inf],
        [0.8520, 0.6040, 0.5259,   -inf]])
```

```
'input_ids': tensor([
[13959,1566,12,2968,10,37, 629,19,1627, 1, 0, 0, 0, 0, 0],
[13959,1566,12,2968,10,2018, 1, 0, 0, 0, 0, 0, 0, 0, 0],
[13959,1566,12,2968,10, 100,19, 3, 9, 182, 307,7142,21,2505, 1]]),
'attention_mask': tensor([
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])}

tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])
```

[PAD] em T5 é zero

Esta `attention_mask`:

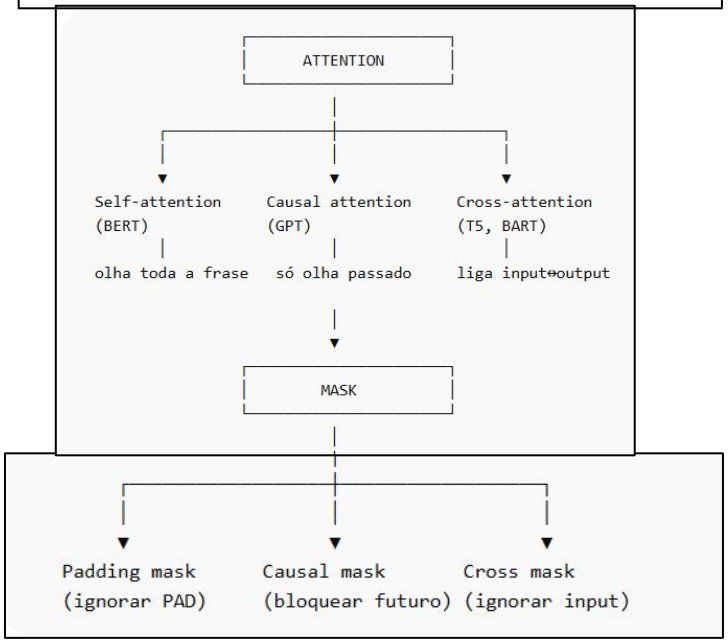
- é usada no encoder
- e também no cross-attention

Tipos de Masks: Masks resumo final

Tipo	Código chave
Padding	mask = [1,1,0,0]
Causal	torch.tril(...)
Cross	usar attention_mask do encoder

Attentions and Masks

Attention and Mask — Mapa Mental



Como interpretar

- Attention → define *como o modelo olha*
- Mask → define *onde o modelo NÃO pode olhar*

Ligações importantes

- Self-attention → usa padding mask
- Causal attention → usa causal mask
- Cross-attention → usa cross mask

Mnemónica:
Attention = escolher
Mask = limitar

FIM